Tema 3.4:

Nuevos paradigmas en LLMs

Minería de textos, Grado en Inteligencia Artificial (GRIA), Universidad de Vigo
© Prof. David Candal Ventureira - 2026

Tabla de contenidos

1 Estado del arte

2 Arquitectura

3 Infraestructura

4 Aplicaciones

1

Estado del arte

Estado del arte: Evolución de los modelos LLM

- Los modelos LLM generativos fueron desarrollados para generar texto coherente.
- Hoy en día, los modelos LLM se utilizan fundamentalmente como **asistentes conversacionales**, con el objetivo de contestar preguntas y completar peticiones del usuario.
  - La **generación aumentada por recuperación (RAG)** permite a los sistemas de aprendizaje automático **acceder a bases de conocimiento externas** en tiempo real, haciendo posible que los sistemas generen respuestas basadas en datos actualizados y no utilizados para el entrenamiento, eliminando así los riesgos de alucinaciones asociados a la información obsoleta.
  - A través de los **agentes**, los modelos LLM son capaces de **ejecutar tareas** externas (ej: llamadas a APIs).

Estado del arte: Evolución de los modelos LLM

- En la actualidad, la industria ya no ve a los LLMs solo como asistentes conversacionales, sino como el **núcleo de control de sistemas complejos**:
  - Integración nativa en procesos industriales.
  - Capacidad de razonamiento auto-adaptable.
  - Sistemas multimodales:
    - Reconocimiento del entorno mediante visión por computadora.
    - Voz como entrada del modelo LLM, sin *speech-to-text*. Análisis de matices emocionales y urgencia.
    - RAG multimodal, basada en texto, imagen y audio.

Estado del arte: Evolución de los modelos LLM

- Algunos nuevos casos de uso definidos son (*edge AI*):
  - Robótica avanzada.
  - Conducción autónoma.
  - Control de procesos industriales (sensores) en tiempo real.
- Estas aplicaciones tienen necesidades específicas, muy diferentes a los servicios actuales:
  - Baja latencia.
  - Disponibilidad *offline* (hacia Internet).
  - Privacidad de los datos.
- A su vez, los dispositivos locales cuentan con limitaciones para ejecutar la inferencia en local:
  - Bajos recursos (cómputo y RAM).
  - Limitaciones de batería.

Estado del arte: Las leyes de escalado

- Las **leyes de escalado** (DeepMind, 2022) definen que, para obtener un modelo con mayor rendimiento, no sólo necesitamos aumentar el número de parámetros y el tiempo de cómputo, si no que también debemos **aumentar proporcionalmente el *dataset* de entrenamiento** (modelo de 70B con mayor rendimiento que un modelo de 280B, al ser entrenado con un *dataset* x4 más grande).
  - 20 *tokens* de entrenamiento por parámetro del modelo.
- Meta ha descubierto que los modelos son capaces de alcanzar **mejor rendimiento con ratios muy superiores**:
  - Llama 3.1 8B: 2000 *tokens* de entrenamiento por parámetro.
  - Durante el entrenamiento, al no disponer de suficientes parámetros como para almacenar todos los patrones lingüísticos:
    - El modelo es incapaz de almacenar más patrones (sube el error de prueba).
    - Llegado a un punto, el modelo detecta reglas lógicas subyacentes al lenguaje, que le permiten **comprimir los patrones** para que “quepan” dentro de los parámetros del modelo.

Estado del arte: El muro de datos

- El rendimiento de un modelo LLM depende en gran medida de los datos con los que es entrenado… Pero **la industria se está quedando sin datos de entrenamiento** (Wikipedia, libros, código de GitHub, artículos científicos…). Esto se conoce como el **muro de datos**.
- Estrategias:
  - Tratar de construir **modelos más eficientes**, conservando el rendimiento original usando menos parámetros.
  - **Generar datos sintéticos**:
    - Generar consultas y posibles respuestas, para problemas matemáticos, lógicos o de programación (verificables).
    - Generar textos bajo diferentes estilos (profesor, ingeniero, médico…).
    - Generar planificaciones para tareas susceptibles de ser consultadas por un humano.
    - Nota: Complementan los datos reales, pero no los sustituyen.

2

Arquitectura

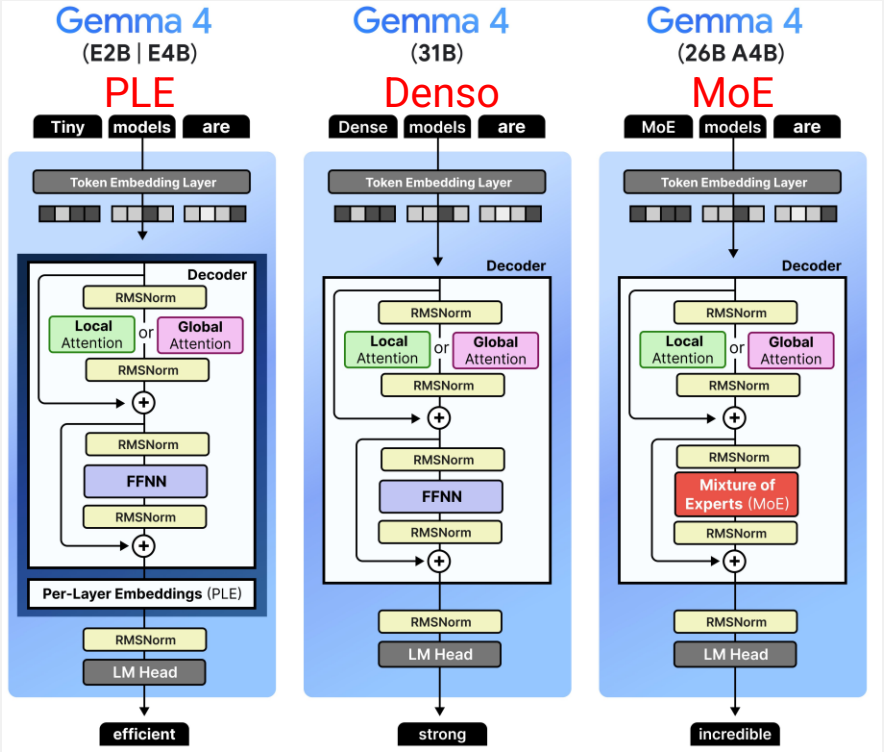
Arquitectura: *Per-layer embeddings*

- La **mezcla de expertos (MoE)** permite dividir las FNNs de cada bloque Transformer en n expertos, permitiendo activar un subconjunto de ellos:
  - No reduce el consumo de memoria.
  - No reduce el coste de cómputo de la atención.

- ***Per-layer embeddings*** **(PLE)** modifica la arquitectura original buscando reducir el número de parámetros activos del modelo durante la inferencia sin deteriorar su rendimiento:
  - Manteniendo la matriz base tradicional a la entrada y salida, **se inyectan adaptaciones de ese *embedding* a la entrada de cada bloque Transformer (PLE)**.
  - El modelo aprende, así, a que **cada capa se enfoque en particularidades semánticas diferentes del *token***. Se refuerzan dominios específicos, y se ajustan los pesos del bloque para tratarlos de manera más adecuada.
  - Los PLEs permiten reducir el número de dimensiones del modelo y sus parámetros efectivos, sin reducir la capacidad para memorizar patrones.

Arquitectura: *Per-layer embeddings*

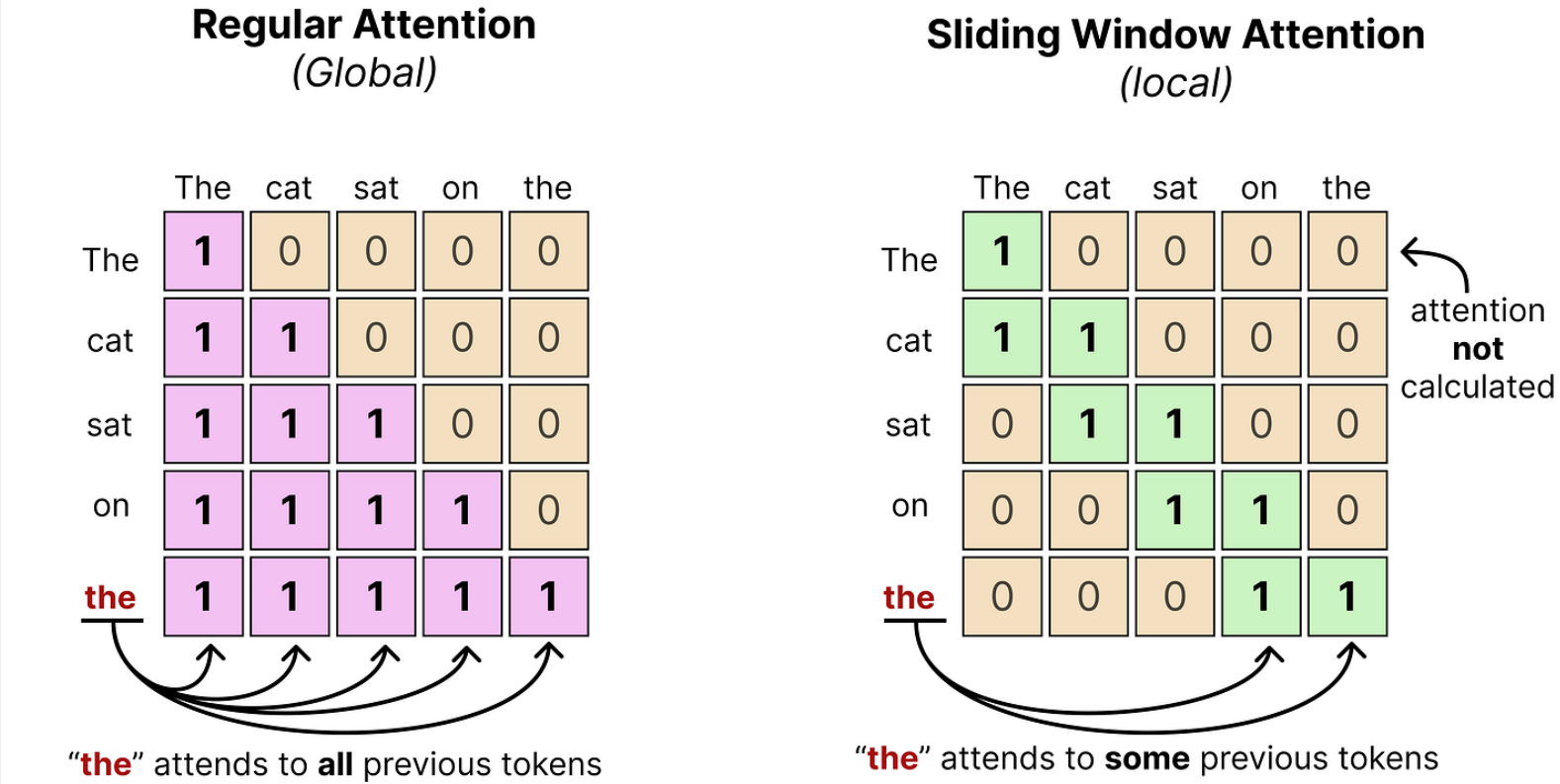
- Comparativa entre un modelo denso vs MoE vs PLE:



Arquitectura: Atención híbrida

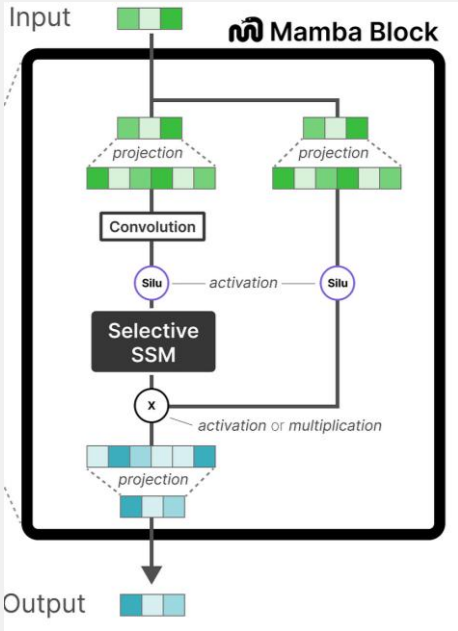
- Con **atención global**, cada *token* “atiende” a **todos los demás *tokens* en el contexto**, lo que tiene un coste de cómputo y memoria cuadráticos ($O(N^2)$).
- Con ***sliding window attention* (SWA, atención local)** cada nuevo *token* sólo analiza **hasta W *tokens* anteriores**, haciendo que el coste de cómputo sea lineal ($O(N)$) y el coste de memoria predecible.
    - Aunque una capa solo vea la información reciente, la información de *tokens* lejanos viaja indirectamente a través de las capas posteriores. Sin embargo, a veces esto no es suficiente cuando los contextos son grandes.
- La **atención híbrida** (ej: Gemma 4) **combina ambos** mecanismos. Estos modelos intercalan capas de atención local, simples, con capas de atención global, de manera que se reduce el coste computacional y de memoria agregado, pero se reintroduce periódicamente la información del contexto completo.

Arquitectura: Atención híbrida



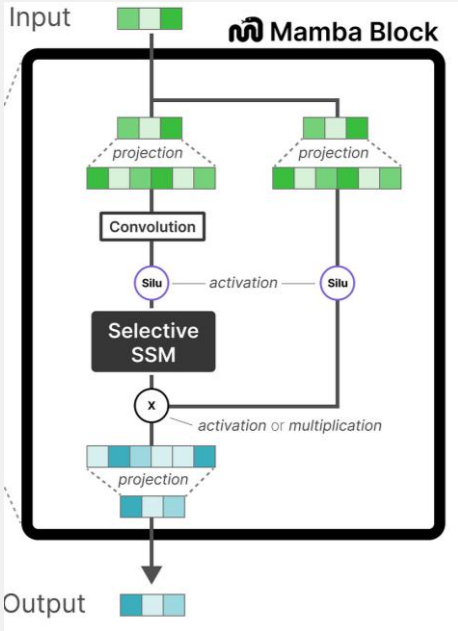
Arquitectura: Arquitectura Mamba

- **Mamba** (2023) es una nueva arquitectura para modelos LLM, basada en un **modelo de espacio de estados (SSM)**, que busca ser una alternativa más eficiente y rápida que la arquitectura Transformer.
  - Todo **el contexto se comprime a un estado de tamaño fijo** (desaparece la caché KV).
  - La **información se procesa de manera secuencial**, con un orden de complejidad lineal (O(N)).
  - Adaptado al HW moderno (evita transferencias de datos entre memorias).



Arquitectura: Arquitectura Mamba

- Ventajas:
  - Ventanas de contexto potencialmente infinitas. Al almacenar el contexto en un estado de tamaño fijo, nunca satura la memoria disponible.
    - Sin embargo, la cantidad de información que se puede comprimir sin pérdida en un espacio finito, es también finita.
  - Mayor velocidad y menor coste.
- Retos:
  - Comprimir todo el contexto puede dificultar la recuperación de información previa si el contexto es largo.
  - Perdida de interpretabilidad (caja negra).



3

Infraestructura

Infraestructura: Paralelismo

- Cuando la combinación de los parámetros del modelo y la caché KV no caben en (V)RAM, debemos **partir la capa Transformer** del modelo en diferentes dispositivos:
  - **Paralelismo de tensores (TP)**: La capa **se divide "horizontalmente"**. Si es posible, los subbloques de atención se dividen por cabezas de atención, y las redes FNN por expertos. Si no existe esa separación, **se dividen las matrices horizontalmente y se combinan tras cada operación**.
    - Rápido y eficiente (los dispositivos operan en paralelo), pero requiere conexiones muy rápidas.
  - **Paralelismo de *pipeline* (PP)**: La capa Transformer **se divide por bloques Transformer**. Cada dispositivo se encarga del procesado de un conjunto de bloques, y le pasa el resultado al siguiente dispositivo para que continúe el procesamiento.
    - No requiere de redes *ad-hoc*, pero los dispositivos operan secuencialmente.

Infraestructura: Paralelismo, *ring attention*

- Cuando la caché KV es muy grande (ej: libro), se recurre al mecanismo ***ring attention***:
  - En lugar de dividir la capa Transformer, **dividimos la caché KV**.
  - Durante el ***prefill***, para cada bloque:
    - Se divide el contexto en D (num. dispositivos) segmentos.
    - **Se procesa cada segmento por el subbloque de atención de cada dispositivo**, generando la caché KV local de cada segmento y una salida parcial.
    - Para calcular la salida final de los *tokens* del segmento, **los dispositivos se transfieren en anillo la caché KV del dispositivo anterior**. El dispositivo calcula la atención de la *query* los *token* locales sobre las *keys* del segmento recibido, ajusta los pesos de atención modificando el softmax, y genera una salida parcial actualizada.
    - Tras recibir las cachés KV de todos los dispositivos anteriores en el anillo, se genera la salida final del bloque Transformer, que pasa a la FNN.
    - Se repite el proceso, usando la salida del bloque como la entrada del siguiente.
  - En ***decoding***:
    - **Cada dispositivo tiene un segmento de la caché KV**. Para la generación de cada nuevo *token*, **se pasa el último *token* generado por todo el anillo, recogiendo la atención sobre cada segmento**, para cada bloque Transformer.

Infraestructura: Cuantización

- La caché KV crece linealmente (se añade una fila en cada matriz K y V). Contextos muy largos pueden llegar a saturar la (V)RAM.
- Adicionalmente, el ancho de banda de la memoria suele a ser el factor más limitante durante la inferencia.
- Los sistemas actuales de **cuantización** comprimen la caché KV de FP16 a FP8/INT8 mediante un **escalado lineal**, con una pérdida de calidad asumible (50% de memoria).
  - Para cada nuevo *token*, se toman los valores mínimo y máximo de todas sus dimensiones, y se escalan todas sus dimensiones en base a ellos.
  - Si un *token* genera vectores k y v que contienen valores muy altos en alguna dimensión, **la pérdida de resolución provoca mucho ruido en el resto de dimensiones**.
  - El ruido generado al bajar a resoluciones menores (ej: INT4) **se magnifica al aplicar operaciones exponenciales** (softmax), transfiriéndose a los *embeddings* y vectores k y v de los nuevos *tokens*. Esto **limita la resolución mínima** con la cual podemos operar.

Infraestructura: Cuantización, TurboQuant

- **TurboQuant** (Google, 2025) es una tecnología de compresión que permite comprimir la caché KV, permitiendo contextos mucho más grandes con una pérdida de precisión despreciable:
  - Se define una **matriz de rotación R**, ortogonal (reversible). Esta matriz está adaptada al modelo (*offline*), de manera que **el producto de k y v por R distribuye los valores de las dimensiones que suelen contener *ourliers*** hacia otras.
  - Cada vez que se calculan los k y v de un nuevo *token*, se rotan a través de R, permitiendo cuantizaciones de hasta INT3 con un error mínimo.
  - Para reducir el error tras la cuantización, se calcula el error de cuantización (al pasar de FP16 a INT3), y se realiza el producto escalar de este valor por un vector de referencia *r*. En un **4to bit se almacena el signo de esta operación** (1 si positivo, 0 si negativo). Al recuperar el dato, tras descuantizar, **se suma o resta** (en función del valor del 4to bit) **un vector γ, precalculado offline**.
  - Como R es reversible, podemos calcular $\boldsymbol{Q \cdot K^T = (Q \cdot R) \cdot (K_{rot} \pm \gamma r)^T}$, donde K_{rot} es la matriz K rotada descuantizada. La operación se realiza en FP16 (aplicamos la rotación a Q, más rápido por ser un vector).

Infraestructura: Cuantización

- Comprimiendo la caché KV, conseguimos:
  - Manejar **contextos más grandes**, con el mismo HW.
  - Reducir el "**muro de memoria**" en las LLMs. La velocidad de inferencia está limitada por la velocidad del bus de memoria, reducir por 4 el tamaño de las trasferencias aumenta en gran medida la generación de *tokens* por segundo.
  - Al reducir el tamaño de la caché KV, podemos **atender a más usuarios de manera simultánea (en *batch*)**.
  - Técnica ***post-training*** (no requiere entrenamiento).

Infraestructura: Decodificación especulativa

- En sistemas comerciales (ej: ChatGPT, Gemini, Grok…), las consultas de los usuarios se procesan en *batches*, aprovechando los recursos de la GPU o dispositivo de inferencia.
- En sistemas privados, la generación autorregresiva provoca que la GPU quede infrautilizada durante la inferencia.
- La **decodificación especulativa** trata de mejorar la eficiencia de estos sistemas. Para ello, se recurre al siguiente proceso:
  - A través de un **modelo más simple** (ej: 1B) **se predicen los siguientes N *tokens***.
  - Creamos un **vector de entrada**, incluyendo el último *token* generado y los N *tokens* seleccionados por el modelo simple.
  - Se realiza ***prefill* del vector de entrada**. En este caso, no descartamos los logits de cada uno de estos *tokens*, sino que **los utilizamos para realizar una selección de siguiente *token***. Se conserva la caché KV resultante.

Infraestructura: Decodificación especulativa

- La decodificación especulativa permite al modelo base **generar, en una única inferencia, múltiples *tokens* de manera simultánea**:
  - Si el modelo simple y el modelo base están alineados, coincidirán habitualmente a la hora de generar texto.
    - Si **los primeros k *tokens*** seleccionados por el modelo base **coinciden** con aquellos del modelo simple (el *token* k+1 no coincide), éstos **se consideran válidos**.
    - En el momento en que uno de los *tokens* seleccionados por el modelo simple **no coincida** con el seleccionado por el modelo base, **se descartan todos los *tokens* posteriores** seleccionados por el modelo simple. **La salida será la combinación de todos los *tokens* válidos** (coincidentes hasta ese momento) **y el primer *token* discordante**. La **caché KV se trunca (elimina filas) en base al número de *tokens* no coincidentes**.
  - La **salida de este sistema es la misma que la que generaría el modelo base** por si mismo. La única diferencia es que realiza una inferencia utilizando múltiples *tokens* futuros posibles, con el objetivo de poder generar múltiples *tokens* simultáneamente.

Infraestructura: Destilación de conocimiento

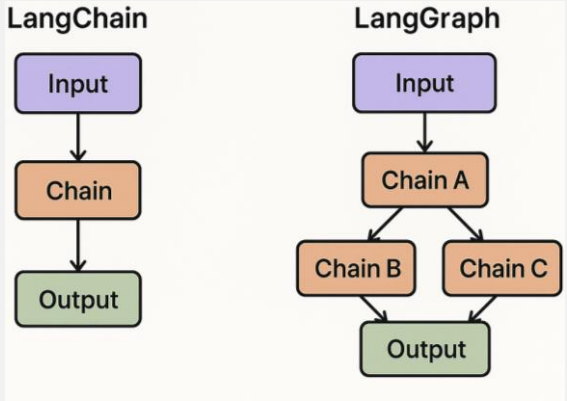
- A través del preentrenamiento estándar, el modelo ajusta sus parámetros en base a la salida esperada (*one-hot*).
  - El modelo debe aprender poco a poco los patrones del lenguaje.
  - Un modelo pequeño puede no ser capaz de capturar patrones complejos debido a su reducido número de parámetros.
- La **destilación de conocimiento** (Google, 2015) tiene como objetivo **entrenar modelos pequeños en base a los resultados de inferencia de un modelo mayor**:
  - Los modelos pequeños no se ajustan en base al *token* seleccionado, sino a la **distribución de logits generada**.
  - Al tratar de replicar las distribuciones de logits, el modelo no sólo aprende simultáneamente el conjunto completo de *tokens* más apropiados para continuar el texto, sino que **este proceso le permite capturar más fácilmente los patrones de lenguaje complejos**.
  - La destilación de conocimiento **suaviza la función de pérdida**.

4

Aplicaciones

Aplicaciones: *Frameworks* industriales

- Los principales *frameworks* industriales para el despliegue de sistemas basados en LLMs son:
  - **LangChain**: Permite implementar agentes y cadenas, basadas en LLMs, y conectarlas con herramientas (ej: RAG).
  - **LangGraph**: Basado en LangChain, permite grafos que contienen bucles y flujos condicionales. Integra funcionalidades adicionales, como intervención humana.



Aplicaciones: Inteligencia en enjambre

- Un LLM modela el lenguaje natural y, dado que el lenguaje codifica conocimiento humano, también captura de forma implícita una aproximación probabilística de ese conocimiento.
- Una **inteligencia en enjambre** es un sistema que combina múltiples agentes basados en LLMs.
  - Cada agente tiene un **perfil (comportamiento)** diferente: escéptico, consumidor, creyente… con diferentes tendencias políticas, niveles de estudios y niveles socioeconómico.
  - La interacción entre estos diferentes perfiles hace **emerger comportamientos en los agentes que no aparecen en ellos por separado**.
- La inteligencia en enjambre requiere una **base de datos relacional** (ej: GraphRAG), de manera que les permita relacionar información y establecer relaciones causales (por qué).
- Aplicaciones: Geopolítica, *marketing*, economía…

Aplicaciones: Inteligencia en enjambre

- **MiroFish** busca crear un **gemelo digital de la sociedad**.
- En lugar de utilizar fórmulas matemáticas para predecir el valor de una variable, **se simula el comportamiento de la sociedad en base a sus interacciones sociales**:
    - Se asignan **sesgos, metas y miedos personalizados** a cada agente, creando población diversa y realista.
    - Los **agentes interaccionan en el entorno** de manera natural (trabajan, compran, viajan…), generando datos sintéticos sociales.
    - **Se introduce un evento** (ej: una nueva ley, una crisis, etc), y **se observa la reacción en cadena** de la sociedad ante este evento, en base a los datos sociales generados.

¿Preguntas?